Shopping With Coupons

Date: 2026-11-11

Abstract

We have N items, N kinds of coupons and D dollars, and we want to buy as many items as possible. Each item can be bought with each coupon only once, so the task is to pick the cheapest (item, coupon) pairs. We use a greedy rule: always buy the cheapest pair that is left. We implement it in three ways: a brute force method that sorts all pairs, a heap method that only looks at a few pairs at a time, and a threshold method that uses binary search. All three give the same answers on 1,639 test cases. The heap method and the threshold method both solve the largest inputs in about 30 ms or less, and the threshold method stays fast even when the amount of money becomes much larger.

Contents

1 Introduction **3**

2 Algorithm Specification **3**

2.1 Greedy Idea . . . 3
2.2 Brute Force . . . 3
2.3 Heap Method . . . 3
2.4 Threshold Method . . . 4
2.5 Correctness . . . 5

3 Testing Results **5**

3.1 Method . . . 5
3.2 Correctness Tests . . . 6
3.3 Test Data . . . 6
3.4 Timing Results . . . 7

4 Analysis and Comments **9**

4.1 Time Complexity . . . 9
4.2 Space Complexity . . . 9
4.3 Discussion of Test Results . . . 9
4.4 Limitations and Improvements . . . 10

5 Conclusion **10**

6 References **10**

7 Source Code Appendix **10**

1 Introduction

In the problem "Shopping With Coupons", there are N items on a shopping list and N kinds of coupons, and we have D dollars. We may buy any item many times and use any coupon many times, but a coupon can be used on the same item only once. For example, we may buy item B1 with coupon A1 and then with coupon A2, but not with A1 again. So every (item, coupon) pair is one possible purchase. It costs the price of the item minus the value of the coupon, and it can be made at most once. The largest coupon is smaller than the lowest price, so every purchase costs at least 1 dollar. We want to buy as many items as possible, and with that number of items we want as much money left as possible. As in the example of the problem, we assume that every purchase uses one coupon, so there are at most N^2 purchases.

The input gives N ($\leq 10^5$) and D ($\leq 10^6$) on the first line, then N prices on the second line and N coupon values on the third line. All numbers are positive integers up to 10^9. The output is one line with the largest number of items and the largest amount of money left, separated by one space.

The difficulty is the number of pairs. With $N = 10^5$ there are 10^{10} pairs, and storing all their costs would need about 80 GB, so we can not simply list them and sort them. However, every purchase costs at least 1 dollar, so with $D \leq 10^6$ we can never buy more than 10^6 items. So we need a way to find the cheapest pairs without looking at all of them.

2 Algorithm Specification

2.1 Greedy Idea

All three methods follow the same rule: buy the cheapest pair that has not been used, and repeat until the cheapest pair costs more than the money we have left. This is right because the pairs do not affect each other: buying one pair does not change the cost of any other pair. So to buy many items we should always pay as little as possible for each one. The proof is in the Correctness section. The three methods only differ in how they find the cheapest pairs.

2.2 Brute Force

The simplest way is to compute the cost of all N^2 pairs, sort them from low to high, and buy from the start of the sorted list until the next pair is too expensive. This is easy to write and easy to trust, so we use it to check the other two methods. But it needs N^2 numbers in memory, so our program only allows it for $N \leq 5000$.

2.3 Heap Method

We do not need all the pairs in sorted order, only the cheapest ones. To find them, we first sort the prices from low to high and the coupons from high to low. Now think of a table where row i is

the i-th cheapest item and column j is the j-th largest coupon. The entry is the cost of that pair, $p_i - c_j$. For the sample (prices 10, 12, 15, 20 and coupons 9, 8, 7, 6) the table is:

Price	Coupon 9	Coupon 8	Coupon 7	Coupon 6
10	1	2	3	4
12	3	4	5	6
15	6	7	8	9
20	11	12	13	14

The costs grow from left to right in every row, and from top to bottom in every column. So the cheapest unused pair of a row is always the first unused pair of that row, and a row can not hold the cheapest pair before the first pair of the row above has been bought. This means we only need to watch one pair per row, and only for the rows we have started.

We keep these pairs in a min-heap, which gives the smallest element and takes a new element in $O(\log n)$ time each [1]. The steps are:

1. **Start.** Put the top-left pair into the heap. It is the cheapest pair of all.
2. **Buy.** Take the cheapest pair out of the heap. If it costs more than the money left, stop. Otherwise pay for it and count one more item.
3. **Add new pairs.** Put the next pair of the same row into the heap. If the pair we bought was the first pair of its row, also put the first pair of the next row into the heap. Then go back to step 2.

For the sample with $D = 30$, the heap gives the costs 1, 2, 3, 3, 4, 4, 5, 6 in this order. They add up to 28. The next cheapest pair costs 6, but only 2 dollars are left, so the answer is 8 items and 2 dollars.

2.4 Threshold Method

The heap method does one step for each item we buy. If the costs are very small, this can be 10^6 steps or more. The threshold method avoids this. The idea is that the greedy rule buys all pairs up to some cost t, and then some of the pairs that cost exactly $t + 1$. If we can find t, we do not need to visit the pairs one by one.

1. **Count for a given t.** For an item with price p, the pairs with cost at most t are the ones whose coupon is at least $p - t$. Since the coupons are sorted from high to low, these are the first k coupons for some k. Their total cost is $k \cdot p$ minus the sum of the k largest coupons, which we read from a prefix sum array. When we go to a more expensive item, k can only get

smaller, so one pointer that only moves in one direction handles all items in $O(N)$ time. If the total passes D we stop at once.

2. **Binary search.** The total cost of all pairs with cost at most t can only grow with t. So we use binary search to find the largest t whose total is at most D. We search between 0 and D, because a pair that costs more than D can never be bought.
3. **Spend the rest.** After paying for all pairs up to t, we use the money left to buy pairs of cost $t + 1$, as many as we can pay for.

For the sample, the pairs with cost at most 5 are 1, 2, 3, 3, 4, 4, 5, with total 22. With cost at most 6 the total would be 34, which is more than 30. So $t = 5$ and 8 dollars are left. One more pair of cost 6 can be bought, which gives 8 items and 2 dollars again.

2.5 Correctness

The greedy rule. Sort the costs of all pairs: $a_1 \leq a_2 \leq \ldots$ Any k different pairs cost at least $a_1 + \cdots + a_k$ in total. Let k be the largest number with $a_1 + \cdots + a_k \leq D$. Then no plan can buy $k + 1$ items, because even the $k + 1$ cheapest pairs cost more than D. And among all plans with k items, the k cheapest pairs cost the least, so they leave the most money. The greedy rule buys exactly these k pairs: it stops at the first pair it can not pay for, and all later pairs cost at least as much.

The heap method. We need to show that the heap always holds the cheapest unused pair. Take any unused pair and walk left in its row to the first unused pair of that row. This pair costs no more. If it is not in the first column, the pair on its left has been bought, so step 3 has put it into the heap. If it is in the first column, walk up the first column to the first unused pair. This pair again costs no more, and it is in the heap, because it is either the top-left pair or the pair above it has been bought. So some pair in the heap is at least as cheap as any unused pair. Every pair enters the heap at most once, so no pair is bought twice.

The threshold method. Let t be the value found by the binary search. All pairs with cost at most t are cheaper than all other pairs, and we can pay for all of them, so the greedy rule buys them all. The total for $t + 1$ is more than D, so there are more pairs of cost $t + 1$ than we can pay for with the money left. So the greedy rule buys as many of them as the money allows, and then stops. This is exactly what step 3 does. If all pairs can be bought, step 3 is skipped.

3 Testing Results

3.1 Method

All tests were run on Linux with an Intel Xeon CPU at 2.10 GHz (4 cores), GCC 13.3.0 and Python 3.13. We compile with `-O2`, and also with AddressSanitizer and UndefinedBehaviorSanitizer to find memory errors.

The program takes the name of a method, so one input can be given to all three methods, and they must print exactly the same line. We also check them against two short Python programs. The first tries every subset of pairs and keeps the best one; it does not use the greedy rule at all, but it only works for $N \leq 3$. The second sorts all pairs in Python and is used for larger inputs.

3.2 Correctness Tests

Test group (count)	Result	Purpose
Hand-made cases (12)	PASS	The sample; one pair that is too expensive, that fits exactly, and that can only be bought once; buying every pair with no money left; one dollar short of buying every pair; all costs equal; $D = 1$; values near 10^9. The answers were worked out by hand.
Tiny inputs (500)	PASS	$N \leq 3$, compared with all subsets of pairs. This checks the greedy rule itself, not only the code.
Random small inputs (1,000)	PASS	$N \leq 40$, compared with the Python sort. Small values give many equal costs; large values go up to 8×10^8.
Generated inputs (96)	PASS	All four data types with $N = 1, 2, 50, 300$ and $D = 1, 10^3, 10^6$, compared with the Python sort.
Large inputs (11)	PASS	$N = 4000$: all three methods must agree. $N = 10^5$ and $D = 10^6$ (the limits of the problem): the heap and threshold methods must agree; one type buys 10^6 items and one type buys nothing.
Bad inputs (4)	PASS	Empty input, $N = 0$, missing numbers and a letter in place of a number must give an error.
Same data everywhere (16)	PASS	The generator must print exactly what a Python version of the standard random engine gives, so the data does not depend on the compiler.

All 1,639 cases pass. The sanitizer build runs a smaller set of 354 cases and reports no error. The timing script below also checks that the methods agree on every input it measures.

3.3 Test Data

Large inputs come from our generator `code/gen.cpp`. It takes N, D, a type and a seed, and makes four types of data:

1. **random**: coupons from 1 to 10^6 and prices from $10^6 + 1$ to 2×10^6. The costs are spread widely, so only a few thousand pairs are cheap enough to buy.
2. **close**: coupons and prices lie within 2,000 of each other, so costs go from 1 to about 4,000 and many pairs are cheap.
3. **equal**: all prices are the same and all coupons are the same, so every pair costs exactly 1. With $D = 10^6$ we buy 10^6 items, the largest number possible.
4. **far**: every pair costs more than 10^6, so nothing can be bought.

The generator takes its random numbers from `mt19937_64`, whose output is fixed by the C++ standard, and turns them into integers with its own short function. So the same arguments give the same data with any compiler.

3.4 Timing Results

The program has a timing mode: it reads the input, runs one method once as warmup and then 10 more times, and reports the median. Only the method is timed (sorting included), not starting the program or reading the input. Every setting uses three inputs (seeds 1, 2 and 3). The tables give the median of the three medians in milliseconds, and the error bars in the figures go from the fastest to the slowest input. The far type is not timed, because nothing is bought.

Experiment 1: number of items. We fix $D = 10^6$ and let N grow from 250 to 10^5. Brute force is only run up to $N = 4000$. The last column is the number of items bought at $N = 10^5$.

Type	Method	$N = 1000$	$N = 4000$	$N = 10^5$	Items
random	Brute force	48.9	1,112	–	
random	Heap	0.066	0.352	12.3	2,442
random	Threshold	0.071	0.436	13.8	2,442
close	Brute force	36.4	489	–	
close	Heap	0.389	0.890	12.5	135,508
close	Threshold	0.073	0.359	10.5	135,508
equal	Brute force	11.3	304	–	
equal	Heap	16.2	17.1	25.5	1,000,000
equal	Threshold	0.014	0.045	2.43	1,000,000

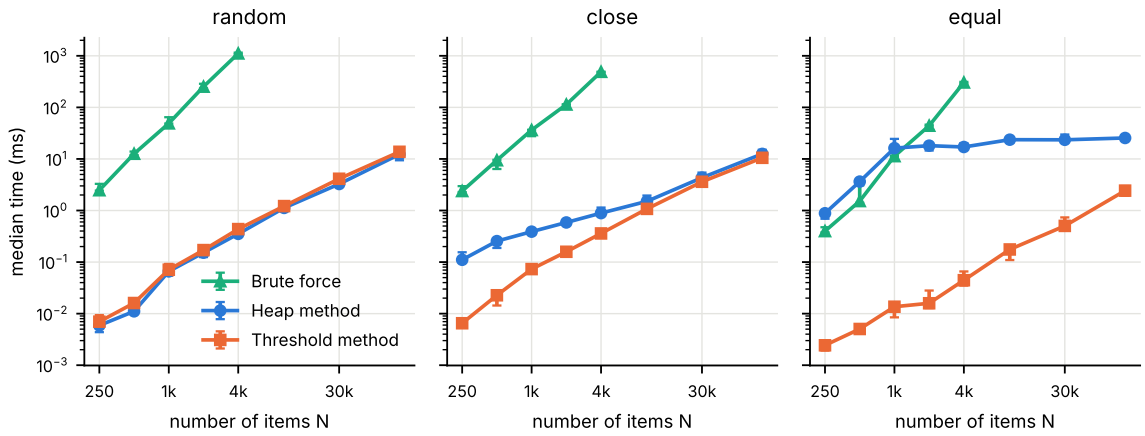


Figure 1: Experiment 1: median time against the number of items, D = 1,000,000

Experiment 2: amount of money. We fix $N = 10^5$ and let D grow from 10^3 to 10^7. The last value is past the limit of the problem; we use it to see the trend. The rows marked "items" give the number of items bought.

Type	Row	$D = 10^4$	$D = 10^5$	$D = 10^6$	$D = 10^7$
close	items	6,102	28,446	135,508	646,022
close	Heap	9.69	8.12	13.1	29.1
close	Threshold	10.4	8.39	12.8	10.7
equal	items	10,000	100,000	1,000,000	10,000,000
equal	Heap	2.17	3.41	27.5	293
equal	Threshold	2.28	2.65	1.90	1.90

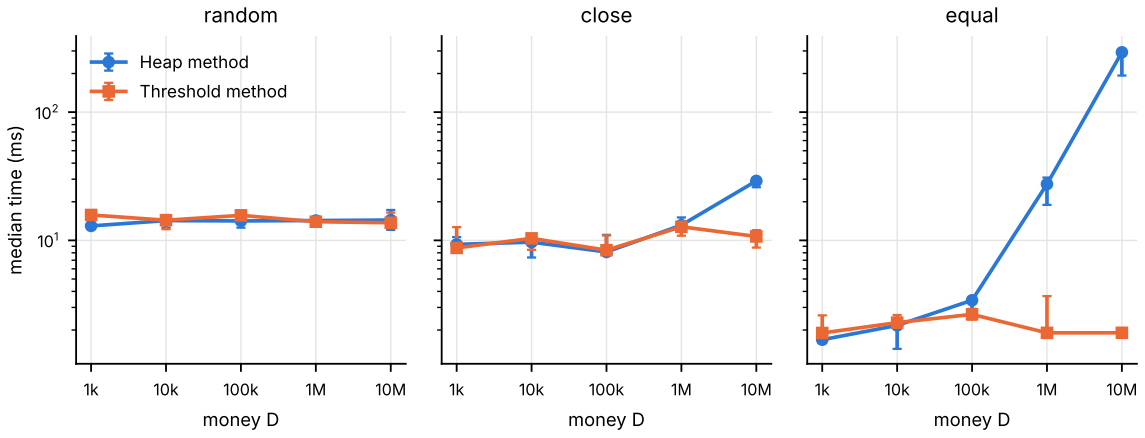


Figure 2: Experiment 2: median time against the amount of money, N = 100,000

4 Analysis and Comments

4.1 Time Complexity

Let K be the number of items we buy. Every purchase costs at least 1 dollar and every pair is used at most once, so $K \leq \min(N^2, D)$.

Brute force. It builds N^2 costs and sorts them, so it takes $O(N^2 \log N)$ time.

Heap method. Sorting the prices and the coupons takes $O(N \log N)$. Each purchase takes one pair out of the heap and puts in at most two, and each heap operation costs $O(\log n)$ for a heap of size n [1]. The heap holds at most one pair per row, so its size is at most N. The total time is $O(N \log N + K \log N)$. With the limits of the problem, $K \leq 10^6$.

Threshold method. After sorting, one count for a given t takes $O(N)$, because the pointer only moves in one direction. The binary search needs about $\log_2 D$ counts, which is 20 for $D = 10^6$. The total time is $O(N \log N + N \log D)$. It does not depend on K.

4.2 Space Complexity

Brute force needs $O(N^2)$ space for the costs. The heap method needs $O(N)$ for the two sorted arrays and the heap. The threshold method needs $O(N)$ for the sorted arrays and the prefix sums.

4.3 Discussion of Test Results

Brute force is only possible for small inputs. In Experiment 1 on random data, its time grows 4 to 5 times each time N doubles, as $N^2 \log N$ says, and it already takes 1.1 seconds at $N = 4000$. The other two methods need less than half a millisecond there.

On random data the heap method and the threshold method take almost the same time, about 12 to 14 ms at $N = 10^5$. Only 2,442 items are bought, so both methods spend nearly all their time on sorting the two arrays, which is the $N \log N$ part they share.

The equal type shows the difference between them. Here $K = 10^6$, so the heap method does 10^6 purchases and needs 16 to 26 ms for every $N \geq 1000$. The threshold method needs only 2.4 ms at $N = 10^5$, about 10 times less. In Experiment 2 the heap time on equal data grows about 10 times when D grows 10 times (27.5 ms to 293 ms), which matches the $K \log N$ part, while the threshold method stays at about 2 ms. At $D = 10^7$ it is about 150 times faster. The close type lies in between: the heap method is only slower at $D = 10^7$, where it buys 646,022 items.

For the limits of the problem both methods are fast enough: the slowest case we found takes about 30 ms. So we use the heap method as the default, because it is shorter and follows the greedy rule directly. The threshold method is the better choice when the amount of money, and so the number of items, can be much larger.

4.4 Limitations and Improvements

1. The time of the heap method grows with the number of items bought. If D were much larger than 10^6, it would become slow, and the threshold method should be used instead.
2. We assume that every purchase uses a coupon. If buying without a coupon were also allowed, the program would need one more step after all pairs are used: spend the rest of the money on the cheapest item at full price.
3. All timings come from one machine with three inputs per setting. On most inputs sorting takes most of the time, so small differences between the two fast methods are within the noise.

5 Conclusion

Buying as many items as possible means buying the cheapest (item, coupon) pairs first. Sorting all pairs is correct but only works for small inputs. After sorting the prices and the coupons, the costs form a table that grows along rows and columns, so a min-heap with one pair per row finds the cheapest pairs in order, and a binary search on the cost finds the answer without visiting the pairs at all. All three methods agree on every test. The heap method is simple and fast enough for the problem, and the threshold method is faster when many items can be bought.

6 References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022, ch. 6. Publisher.

7 Source Code Appendix

The source code is in the `code/` folder of the submission. `code/main.cpp` is the solver: `solve_brute`, `solve_heap` and `solve_search` are the three methods, and `main` reads the input, runs the chosen method and can also time it. `code/gen.cpp` is the test data generator. The `README.md` shows how to compile and run them.